

Programmation fonctionnelle 2
Preuves et programmes
CM5 : types variants et inductifs rékursifs

Pierre Rousselin

Université Sorbonne Paris Nord

février 2026

Listes en OCaml

Listes et entiers naturels de Peano en Rocq

Plus de listes, de nat et de preuves par induction

Terme de preuve pour l'induction

Listes en OCaml

- ▶ Les constructeurs des variants OCaml peuvent avoir des arguments dont le type est celui qui est construit.
- ▶ `type 'a ma_liste = Nil | Cons of 'a * 'a ma_liste`
- ▶ Ce genre de définition est appelée « inductive ».
- ▶ `let one_two = Cons (1, Cons (2, Nil))
(* val one_two : int ma_liste = Cons (1, Cons (2 Nil)) *)`
- ▶ un constructeur `Nil` sans argument pour la liste vide de n'importe quel type
- ▶ un constructeur `Cons` qui prend deux arguments :
 - ▶ un élément en **tête** (*head*) de liste
 - ▶ une liste d'éléments du même type appelée **queue** (*tail*) de la liste

Listes en OCaml

- ▶ Les constructeurs des variants OCaml peuvent avoir des arguments dont le type est celui qui est construit.
- ▶ `type 'a ma_liste = Nil | Cons of 'a * 'a ma_liste`
- ▶ Ce genre de définition est appelée « inductive ».
- ▶ `let one_two = Cons (1, Cons (2, Nil))
(* val one_two : int ma_liste = Cons (1, Cons (2 Nil)) *)`
- ▶ un constructeur `Nil` sans argument pour la liste vide de n'importe quel type
- ▶ un constructeur `Cons` qui prend deux arguments :
 - ▶ un élément en **tête** (*head*) de liste
 - ▶ une liste d'éléments du même type appelée **queue** (*tail*) de la liste
- ▶ Pour les « vraies » listes OCaml, elles sont définies par :
`type 'a list = [] | (:::) of 'a * 'a list`
- ▶ La liste vide (qu'on appelle bien « nil » à l'oral) est notée `[]` et
- ▶ l'autre constructeur (qu'on appelle bien « cons » à l'oral) est donné sous la forme d'un opérateur qui prend la tête (élément) à gauche et la queue (liste) à droite

Notations pratiques

Voici plusieurs façons d'écrire la liste qui contient (dans cet ordre) 42, 10 et 25

- ▶ En écrivant à la main les cons, sans la syntaxe des opérateurs

```
let 11 = ( :: ) (42, ( :: ) (10, ( :: ) (25, [])))
```

- ▶ En utilisant :: comme un opérateur (ouf!)

```
let 12 = 42 :: 10 :: 25 :: []
```

- ▶ En utilisant la notation [...; ...] :

```
let 13 = [42; 10; 25]
```

- ▶ Attention à l'erreur courante :

```
let 14 = [42, 10, 25]
```

(liste à un seul élément qui est un triplet... *)*

Définition de fonctions

On définit les fonctions comme toujours avec un filtrage de motif :

- ▶ prédicat `is_empty` sur les listes :

Définition de fonctions

On définit les fonctions comme toujours avec un filtrage de motif :

- ▶ prédicat `is_empty` sur les listes :

```
let is_empty l =  
  match l with  
  | [] -> true  
  | _ -> false
```

- ▶ longueur d'une liste :

Définition de fonctions

On définit les fonctions comme toujours avec un filtrage de motif :

- ▶ prédicat `is_empty` sur les listes :

```
let is_empty l =  
  match l with  
  | [] -> true  
  | _ -> false
```

- ▶ longueur d'une liste :

```
let rec length l =  
  match l with  
  | [] -> 0  
  | h :: q -> 1 + length q
```

- ▶ mettre une liste à la suite d'une autre

Définition de fonctions

On définit les fonctions comme toujours avec un filtrage de motif :

- ▶ prédicat `is_empty` sur les listes :

```
let is_empty l =  
  match l with  
  | [] -> true  
  | _ -> false
```

- ▶ longueur d'une liste :

```
let rec length l =  
  match l with  
  | [] -> 0  
  | h :: q -> 1 + length q
```

- ▶ mettre une liste à la suite d'une autre

```
let rec append l0 l1 =  
  match l0 with  
  | [] -> l1  
  | h :: q -> h :: (append q l1)
```

Réversivité, méthode de base

- ▶ Avant de chercher éventuellement des choses plus compliquées,
- ▶ toujours commencer par un **match sur les constructeurs** (et d'abord sur une seule variable, s'il y en a plusieurs),
- ▶ si on peut conclure directement (sans réversivité), tant mieux
- ▶ sinon, par exemple pour **append** juste avant, se demander comment reconstruire la solution si jamais je l'ai déjà pour `append q 11`.

Listes en OCaml

Listes et entiers naturels de Peano en Rocq

Plus de listes, de nat et de preuves par induction

Terme de preuve pour l'induction

Listes

Ce qu'on a dans le prélude :

- ▶ **Inductive** list (A : **Type**) : **Type** :=
 nil : list A | cons : A -> list A -> list A.
- ▶ **Check** list. (* list : Type -> Type *)
 Check list bool. (* list bool : Set *)
- ▶ On a bien une fonction qui à un type associe un (nouveau) type.

Listes

Ce qu'on a dans le prélude :

- ▶ **Inductive** list (A : **Type**) : **Type** :=
 nil : list A | cons : A -> list A -> list A.
- ▶ **Check** list. (** list : Type -> Type **)
 Check list bool. (** list bool : Set **)
- ▶ On a bien une fonction qui à un type associe un (nouveau) type.
- ▶ Les constructeurs sont de type dépendants :
 About nil. (** nil : forall {A : Type}, list A **)
 (** les accolades dans {A} montrent que c'est un arg implicite **)
 Check nil. (** list ?A, car l'argument de type est implicite **)
 Check @nil bool. (** list bool **)
- ▶ Idem pour cons
 About cons. (** forall {A : Type}, A -> list A -> list A **)
 Check cons. (** ?A -> list ?A -> list ?A **)
 Check @cons bool. (** bool -> list bool -> list bool **)

Listes et notations

- ▶ Exemple d'inférence de type :

```
Check (cons 42 (cons 10 (nil))).  
(* 42 :: 10 :: nil : list nat *)
```

- ▶ C'est tout de même plus agréable à écrire que :

```
Check (@cons nat 42 (@cons nat 10 (@nil nat))).  
(* 42 :: 10 :: nil : list nat *)
```

- ▶ Au passage, on voit que la notation `::` est également disponible, on se renseigne :

- ▶ Locate `"::"`.

```
(* Notation "x :: y" := (cons x y) *)  
Print Notation "_ :: _".  
(* Notation "_ :: _" at level 60, right associativity. *)
```

Listes et notations (2)

- Pour obtenir les autres notations à la OCaml, on charge et on importe le module `List` de la bibliothèque standard, puis on importe son sous-module `ListNotations` :

```
From Coq Require Import List.  
Import ListNotations.
```

```
About "[ ]".  
Print Notation "[ ]".
```

```
Locate "[ x ; y ; .. ; z ]".  
Print Notation "[ x ; y ; .. ; z ]".
```

```
Check [42; 10; 25].
```

- On le fait lorsqu'on travaille beaucoup avec des listes.

Fonctions sur les listes

- Équivalent (dans bool) en Rocq de `is_empty`.

```
Definition is_empty {A : Type} (l : list A) :=  
  match l with  
  | [] => true  
  | _ => false  
end.
```

- On sait calculer avec cette définition :

```
Eval simpl in (is_empty []). (* true *)  
Eval simpl in (is_empty [32; 10]). (* false *)
```


Fonctions sur les listes

- L'équivalent de **let rec** (global) en Rocq est **Fixpoint**¹.

```
Fixpoint app {A : Type} (l0 l1 : list A) :=  
  match l0 with  
  | [] => l1  
  | h :: q => h :: (app q l1)  
end.
```

- On sait calculer avec cette définition :

```
Eval simpl in (app [32; 10] [1; 2; 3]).  
(* = [32; 10; 1; 2; 3] : list nat *)
```

- On peut même réduire partiellement, tant qu'on peut :

```
Section ExpleAppend.  
  Context (A : Type) (l1 l2 : list A) (a b : A).  
  Eval simpl in (app (cons a (cons b l1)) l2).
```

- donne :

1. `append` s'appelle `app` dans la bibliothèque standard de Rocq.

Fonctions sur les listes

- L'équivalent de **let rec** (global) en Rocq est **Fixpoint**¹.

```
Fixpoint app {A : Type} (l0 l1 : list A) :=  
  match l0 with  
  | [] => l1  
  | h :: q => h :: (app q l1)  
end.
```

- On sait calculer avec cette définition :

```
Eval simpl in (app [32; 10] [1; 2; 3]).  
(* = [32; 10; 1; 2; 3] : list nat *)
```

- On peut même réduire partiellement, tant qu'on peut :

```
Section ExpleAppend.  
  Context (A : Type) (l1 l2 : list A) (a b : A).  
  Eval simpl in (app (cons a (cons b l1)) l2).
```

- donne :

```
(* = a :: b :: app l1 l2 : list A *)
```

- mais parfois, on ne peut pas réduire :

- **Eval simpl in** (app l1 []).
 (* = app l1 [] : list A *)

```
End ExpleAppend.
```

1. **append** s'appelle **app** dans la bibliothèque standard de Rocq.

app la liste vide (1)

- **Lemma** `app_nil_l {A : Type} : forall (l : list A), app [] l = l.`

Proof.

`intros l.`

`simpl. (* par calcul, (app [] l) se réduit en l *)`

`reflexivity. (* l = l, fin de la preuve *)`

Qed.

- **Lemma** `app_nil_r {A : Type} : forall (l : list A), app l [] = l.`

Proof.

(C'est certainement vrai, allons-y. *)*

`intros l.`

`simpl. (* arg, rien ne se passe *)`

(essayons par cas *)*

`destruct l as [| h q]. (* nil ou h :: q *)`

- `simpl. (* par calcul, app [] [] se réduit en [] *)`

`reflexivity. (* ok, [] = [] *)`

- `simpl. (* par calcul, app (h :: q) [] = h :: (app q []) *)`

(Hmm... ne suffit pas... *)*

(On a besoin de la preuve que app q [] = q *)*

Abort. *(* On recommence avec une preuve par induction. *)*

app la liste vide avec induction

- **Lemma** `app_nil_r` `{A : Type}` : `forall` (`l` : `list A`), `app l []` = `l`.
Proof.

```
  intros l.
  induction l as [| h q IH].
  - (* cas l = [] *)
    simpl.
    reflexivity.
  - (* cas l = h :: q
      avec l'hypothèse d'induction :
      IH : app q [] = q *)
    simpl.
    rewrite IH. (* changer (app q []) en q dans le but *)
    reflexivity. (* h :: q = h :: q *)
```

Qed.

- On a utilisé la tactique **induction**.

Tactique induction

- ▶ La syntaxe est
`induction valeurTypeInductif as`
`[varsEtOuHyps1 | varsEtOuHyps2..]`.
- ▶ les différentes alternatives dans les `[..|..]` correspondent aux différents constructeurs du type de `valeurTypeInductif`.
- ▶ avec des variables liées aux arguments des constructeurs,
- ▶ comme avec `destruct`, mais avec en plus,
- ▶ **pour chaque argument du même type**, une hypothèse, dite « d'induction »
- ▶ disant que le but courant est satisfait pour cette variable.
- ▶ Dans notre cas :

`induction l as [ | h q IH ] .`

- ▶ Premier cas : `nil` (sans argument), d'où le `[ |`
- ▶ Deuxième cas : `cons : A -> list A -> list A`
 - ▶ `h : A`
 - ▶ `q : list A`
 - ▶ `IH : app q [] = q`

Entiers de Peano

- ▶ On n'a pas encore défini `length`
- ▶ car il faut parler des entiers, dits de Peano².
- ▶ Alors, c'est facile, c'est comme `list`, mais sans les éléments.
Dans le prélude on a :
- ▶ `Inductive nat : Set := 0 | S : nat -> nat.`
- ▶ La lettre `0` désigne le constructeur sans argument, le nombre 0.
- ▶ Le constructeur `S`, pour « successeur » permet de construire un nouvel entier à partir de son argument.
- ▶ `0` représente 0, `S 0` représente 1, `S (S 0)` représente 2, ...

2. Giuseppe Peano, 1858 – 1932

Entiers de Peano et notations

- ▶ On a des notations pour la saisie et pour l’affichage en décimal.
- ▶ **Set Printing All.**
Check 4.
`(* S (S (S (S 0))) : nat *)`
Unset Printing All.
- ▶ **Check** (S (S (S (S (S (S 0)))))).
`(* 6 : nat *)`
- ▶ On a des entiers en « unaire »,
- ▶ peu efficaces en mémoire, et pour le calcul,
- ▶ mais parfaits pour écrire et prouver des propositions.

Addition = app

- Définition de l'addition :

Addition = app

- Définition de l'addition :

```
Fixpoint add (n m : nat) :=  
  match n with  
  | 0 => m  
  | S p => S (add p m)  
end.
```

- Ça calcule :

```
Eval simpl in (2 + 2).
```

Addition = app

- Définition de l'addition :

```
Fixpoint add (n m : nat) :=  
  match n with  
  | 0 => m  
  | S p => S (add p m)  
end.
```

- Ça calcule :

```
Eval simpl in (2 + 2).  
(* 4 : nat *)
```

- Ça calcule autant que ça peut :

```
Section AddExple.  
  Context (n m : nat).  
  Eval simpl in (0 + m). (* m *)  
  Eval simpl in (2 + m). (* S (S m) *)  
  Eval simpl in (S (S n) + m). (* S (S (n + m)) *)  
  Eval simpl in (n + 0). (* n + 0 *)  
  Eval simpl in (n + 1). (* n + 1 *)  
End AddExple.
```

Ajouter 0

► `Lemma add_0_1 : forall (n : nat), 0 + n = n.`

Ajouter 0

► `Lemma add_0_l : forall (n : nat), 0 + n = n.`

`Proof.`

`intros n. (* Soit (n : nat). *)`

`simpl. (* On réduit 0 + n en n *)`

`reflexivity. (* n = n, ok *)`

`Qed.`

► `Lemma add_0_r : forall (n : nat), n + 0 = n.`

Ajouter 0

► **Lemma** add_0_l : forall (n : nat), 0 + n = n.

Proof.

```
  intros n. (* Soit (n : nat). *)  
  simpl. (* On réduit 0 + n en n *)  
  reflexivity. (* n = n, ok *)
```

Qed.

► **Lemma** add_0_r : forall (n : nat), n + 0 = n.

Proof.

```
  intros n. induction n as [| p IH].  
  - simpl. (* 0 + 0 se réduit en 0 *)  
    reflexivity. (* 0 = 0 ok *)  
  - (* n = S p où p satisfait  
      IH : p + 0 = p *)  
    simpl. (* S p + 0 se réduit en S (p + 0) *)  
    rewrite IH. (* On utilise IH pour changer  
                p + 0 en p dans le but *)  
    reflexivity. (* S p = S p, ok *)
```

Qed.

Listes en OCaml

Listes et entiers naturels de Peano en Rocq

Plus de listes, de nat et de preuves par induction

Terme de preuve pour l'induction

length, app, etc

- ▶ Longueur d'une liste :

length, app, etc

- Longueur d'une liste :

```
Fixpoint length {A : Type} (l : list A) :=  
  match l with  
  | [] => 0  
  | _ :: q => S (length q)  
end.
```

- Si $l, l' : \text{list } A$ (pour un type A quelconque), que vaut $\text{length } (\text{app } l \ l')$?

length, app, etc

- Longueur d'une liste :

```
Fixpoint length {A : Type} (l : list A) :=  
  match l with  
  | [] => 0  
  | _ :: q => S (length q)  
end.
```

- Si $l, l' : \text{list } A$ (pour un type A quelconque), que vaut $\text{length } (\text{app } l \ l')$?

```
Lemma length_app {A : Type} (l l' : list A) :  
  length (app l l') = length l + length l'.
```

Proof.

```
  induction l as [| h q IH].
```

```
  - simpl. reflexivity.
```

```
  - simpl. rewrite IH. reflexivity.
```

Qed.

Associativité

Que dire de $\text{app} (\text{app } 11 \ 12) \ 13$?

Associativité

Que dire de `app (app l1 l2) l3`?

```
Lemma app_assoc {A : Type} (l1 l2 l3 : list A) :  
  app (app l1 l2) l3 = app l1 (app l2 l3).
```

Proof.

```
  induction l1 as [| h q IH].  
  - simpl. reflexivity.  
  - simpl. rewrite IH.  
    reflexivity.
```

Qed.

Listes en OCaml

Listes et entiers naturels de Peano en Rocq

Plus de listes, de nat et de preuves par induction

Terme de preuve pour l'induction

C'est quoi *induction* ?

- ▶ Jusqu'à présent, on a toujours (ou presque) pu voir les preuves comme des programmes.
- ▶ Les preuves par induction n'y échappent pas.
- ▶ Une preuve par induction est un programme récursif.
- ▶ Ce programme³ est généré à la définition des types inductifs.
- ▶ About `nat_ind`.

```
(* forall (P : nat -> Prop),  
    P 0 -> (forall n, P n -> P (S n)) -> forall n, P n *)
```
- ▶ About `list_ind`.

```
(* forall [A : Type] (P : list A -> Prop),  
    P []  
    -> (forall (a : A) (l : list A), P l -> P (a :: l))  
    -> forall l : list A, P l *)
```
- ▶ La tactique **induction** applique (**apply**) ce programme puis fait les **intros** correspondants dans chaque branche.

3. En vrai, il y en a plusieurs à cause des sortes différentes, mais peu importe dans ce cours.

C'est quoi *induction* (2)

- Ok, on y va, on essaie d'écrire nous-même ce programme.

C'est quoi induction (2)

- ▶ Ok, on y va, on essaie d'écrire nous-même ce programme.
- ▶ **Fixpoint** my_nat_ind (P : nat -> Prop) (h0 : P 0)
 (hnSn : forall n, P n -> P (S n)) n : P n :=
 match n with
 | 0 => h0
 | S p => hnSn p (my_nat_ind P h0 hnSn p)
 end.
- ▶ La même chose pour les listes :

C'est quoi induction (2)

- ▶ Ok, on y va, on essaie d'écrire nous-même ce programme.

- ▶ **Fixpoint** my_nat_ind (P : nat -> Prop) (h0 : P 0)
 (hnSn : forall n, P n -> P (S n)) n : P n :=
 match n with
 | 0 => h0
 | S p => hnSn p (my_nat_ind P h0 hnSn p)
 end.

- ▶ La même chose pour les listes :

```
Fixpoint my_list_ind {A : Type} (P : list A -> Prop) (hnil : P [])  
  (hcons : forall a l, P l -> P (a :: l)) l : P l :=  
  match l with  
  | [] => hnil  
  | h :: q => hcons h q (my_list_ind P hnil hcons q)  
  end.
```